



INSTITUTO POLITECNICO NACIONAL



ESCUELA SUPERIOR DE CÓMPUTO

Ingeniería en Inteligencia Artificial

ACADEMIA: Sistemas Distribuidos

MATERIA: “Cómputo Paralelo”

GRUPO: 6BM1

ACTIVIDAD:

Práctica 1: Plataformas y herramientas para cómputo paralelo.

PRESENTA:

Dimas Gonzalez Santiago - 2024631046

Mendieta Mondragón Cristian Enrique - 2024630811

Miranda Ferreyra Uriel - 2024630834

PLAN: 2020

FECHA: 1 oct 2026

Profesor de Materia: Rodrigo Ojeda Santillan

Sección 1. Planteamiento de la práctica

Objetivo

El objetivo de esta práctica es establecer que antes de paralelizar hay que saber medir, dado que todo speedup se calcula contra la línea base secuencial y una línea base mal medida invalida todo lo que sigue. Al terminar, se busca preparar y verificar un entorno Linux, describir el hardware sobre el que se va a medir (incluyendo núcleos físicos y lógicos, cachés, memoria, extensiones vectoriales y GPU) y medir tiempos de ejecución de forma reproducible utilizando un reloj correcto, calentamiento, repeticiones, promedio y desviación estándar. Además, se construirá una línea base secuencial para expresarla en GFLOPS y explicar cuánto cambia con el nivel de optimización del compilador y con NumPy, finalizando con la ejecución de código en una GPU para medirlo correctamente mediante sincronización.

Introducción

En el análisis de rendimiento, medir correctamente requiere distinguir entre el tiempo de pared (wall-clock), que es lo que tarda el programa según un reloj externo y se usa para calcular speedup, y el tiempo de CPU, que suma el tiempo de todos los hilos. Como parte del protocolo de medición, es obligatorio realizar una ejecución de calentamiento que se descarta, ya que la primera corrida paga fallos de página, caché fría y subida de frecuencia del procesador. Para esta evaluación, usamos la multiplicación de matrices $C = A \cdot B$ de $n \times n$ en doble precisión. Se utiliza esta operación porque es el núcleo del aprendizaje profundo y porque su costo es exactamente $2n^3$ operaciones de punto flotante, garantizando que los GFLOPS se calculen sin ambigüedad. A través de las corridas del benchmark, se analizará la diferencia entre usar el nivel de optimización del compilador -O0 y -O3 -march=native, así como la razón de tiempos al usar una versión ingenua i-j-k en C, código en Python puro y librerías optimizadas como NumPy (BLAS), lo que permitirá observar el paralelismo oculto cuando NumPy utiliza múltiples hilos sin pedirlo explícitamente. Finalmente, se contrastaron los tiempos de la CPU contra la GPU usando CuPy, comprobando empíricamente que la GPU trabaja de forma asíncrona, por lo que hay que sincronizar antes de leer el reloj para no obtener un número falso.

Sección 2. Entorno y ficha técnica

El equipo optó por el uso de una computadora de escritorio usada a modo de servidor privado para los miembros del equipo. Se accede de forma remota a este mediante *tailscale* donde cada integrante tiene su propio entorno de trabajo pero todos utilizan los mismo recursos, es por eso que las tres tablas son exactamente iguales y dentro de esta se realizaron las mediciones.

Dimas Gonzalez Santiago, Mendieta Mondragon Cristian Enrique & Miranda Ferreyra Uriel

Campo	Valor
Modelo de la CPU	AMD Ryzen 5 3600 6-Core Processor
Nucleos fisicos/ hilos logicos	6 / 12
Frecuencia base/ turbo	2.20 GHz / 4.20 GHz

Cache L1d/L2/L3	L1d: 192 KiB (6 instancias) / L2: 3 MiB (6 instancias) / L3: 32 MiB (2 instancias)
Memoria RAM	31 GiB (Libre: 23 GiB)
Extensiones vectoriales	AVX, AVX2, FMA, SSE4_2
Sistema operativo y kernel	Ubuntu 24.04.5 LTS
Compilador	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Python/NumPy/BLAS	Python: 3.10.12 / NumPy: 2.2.6 / BLAS: OpenBLAS 0.3.29
GPU	NVIDIA GeForce RTX 3060 Ti (Driver: 580.178.04, CUDA: 13.0)
Condiciones	PC de escritorio, gobernador de CPU schedutil, sin otras aplicaciones pesadas abiertas durante la ejecución.

Tabla 1. Hardware equipo compartido.

```

entorno.txt

gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
g++ (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Python 3.10.12
numpy 2.2.6
git version 2.34.1
#define _OPENMP 201511

Hola desde el hilo 0 de 1
Hola desde el hilo 0 de 4
Hola desde el hilo 2 de 4
Hola desde el hilo 3 de 4
Hola desde el hilo 1 de 4

```

Figura 1. Salida obtenida de la verificación y prueba del toolchain.

Sección 3. Metodología de medición

Para aislar el rendimiento real del hardware y realizar un cálculo de aceleración (*speedup*) válido, se distinguió entre el tiempo de pared (*wall-clock*) y el tiempo de CPU. Se empleó la función *clock_gettime(CLOCK_MONOTONIC)* en C y *time.perf_counter()* en Python para medir el tiempo transcurrido de forma aislada, descartando el uso de *time.time()* posibles saltos o desfases por ajustes en el reloj del sistema. Además, en las pruebas con NumPy se utilizó *time.process_time()* con el fin de registrar el tiempo exclusivo de CPU y detectar posibles paralelismos ocultos. La región cronometrada se asignó únicamente al cómputo central, excluyendo los tiempos de reserva de memoria e inicialización de estructuras. Para evitar que el compilador descarte las operaciones mediante optimización de código muerto (*dead-code elimination*), se imprimió una suma de verificación (*checksum*) con el resultado de las matrices. Siguiendo el protocolo experimental, cada prueba inició con una ejecución de calentamiento (*warm-up*) que fue eliminada para prevenir fallos de página, penalizaciones por llenado de caché fría y retrasos dado el escalado de frecuencia del procesador. Posteriormente, se registraron al menos 5 repeticiones independientes para reportar el promedio y la desviación estándar ($\bar{x} \pm \sigma$). Como criterio de estabilidad y control de variabilidad, se definió que ante una dispersión relativa mayor al 10% ($\sigma > 10\%$), se investigaría la causa de la anomalía. Todas las pruebas se ejecutaron en el entorno remoto compartido con el gobernador de CPU configurado en modo *schedutil* y sin aplicaciones en segundo plano, ejecutando la compilación vía *make* y empleando

el script bash *scripts/run_all.sh* para la automatización y reproducibilidad de los resultados. A continuación se muestran tanto el script para lanzar las pruebas como las banderas declaradas en el *Makefile*.

```
Makefile

CC = gcc

all: matmul_00 matmul_02 matmul_03 hello_omp

matmul_00: matmul.c
$(CC) -O0 -std=c11 $< -o $@

matmul_02: matmul.c
$(CC) -O2 -std=c11 $< -o $@

matmul_03: matmul.c
$(CC) -O3 -march=native -std=c11 $< -o $@

hello_omp: hello_omp.c
$(CC) -fopenmp $< -o $@

clean:
rm -f matmul_00 matmul_02 matmul_03 hello_omp
```

Figura 2. Comandos de compilación utilizados (Makefile).

```
Script de Ejecución

set -e; mkdir -p ../results; make -s

echo "variant,n,rep,seconds,gflops,checksum" > ../results/c.csv
for n in 256 512 1024; do
  for v in 00 02 03; do
    ./matmul_$v $n 5 c_$v >> ../results/c.csv
  done
done

echo "variant,n,rep,seconds,gflops,checksum" > ../results/py_puro.csv
for n in 64 128 256; do
  python3 matmul_pure.py $n 5 >> ../results/py_puro.csv
done

echo "variant,n,rep,seconds,gflops,cpu_over_wall" > ../results/numpy.csv
for n in 256 512 1024 2048; do
  python3 matmul_numpy.py $n 5 >> ../results/numpy.csv
  OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 \
  python3 matmul_numpy.py $n 5 | sed 's/^numpy/numpy_1hilo/' >>
  ../results/numpy.csv
done
```

Figura 3. Script lanzador de pruebas empleado.

Sección 4. Resultados

variant	n	t_mean	t_std	gflops
c_00	256	0.0457794	0.0001163907213	0.733
c_00	512	0.3831076	0.0005666527155	0.7006
c_00	1024	3.1317276	0.008198412151	0.6858
c_02	256	3.45E-02	6.43E-05	0.9734
c_02	512	0.7408724	0.00693981025	0.3624
c_02	1024	5.9702006	0.01445718909	0.3598
c_03	256	3.31E-02	7.61E-05	1.015
c_03	512	0.7406598	0.001332342561	0.3624
c_03	1024	5.9712582	0.007225108975	0.3594
numpy	256	3.58E-04	8.67E-05	98.7116

numpy	512	0.0035172	0.00170643069	99.1828
numpy	1024	0.0247154	0.007766358368	95.497
numpy	2048	0.069718	0.004385995155	247.1728
numpy_1hilo	256	0.0007466	0.000137285469	46.2744
numpy_1hilo	512	0.0054948	0.0005203697724	49.2144
numpy_1hilo	1024	0.0421246	0.002745680753	51.1416
numpy_1hilo	2048	0.3215238	0.0004985897111	53.4328
py_puro	64	1.50E-02	4.29E-05	0.03504
py_puro	128	0.1246322	0.0004505393435	0.03364
py_puro	256	0.9495878	0.004469163647	0.03534

Tabla 2. Resumen comparativo de los 3 métodos utilizados.

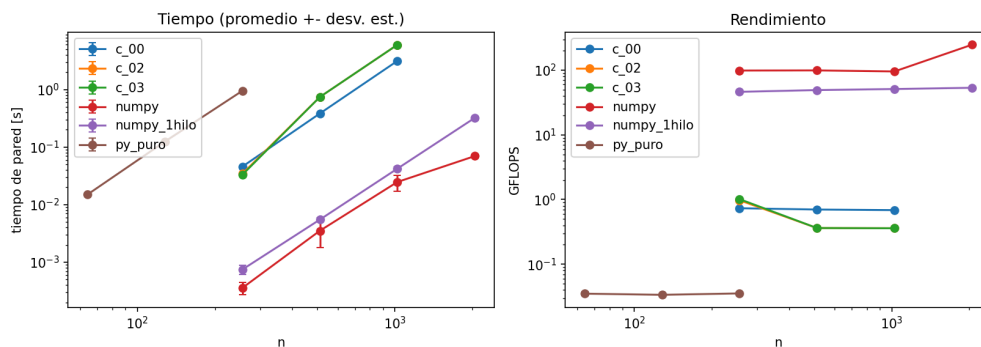


Figura 4. Gráficas comparativas, izquierda, tiempo de pared vs n, derecha, GFLOPs vs n.

Sección 5. Análisis

1. Nivel de optimización

¿Cuántas veces más rápido es -O3 -march=native que -O0 para n = 1024? Para n=1024, la versión optimizada con -O3 -march=native fue más lenta con 5.97s que la versión -O0 de 3.13s dándonos una razón de 0.52x.

Sin paralelismo de hilos, ¿qué está haciendo el compilador? El compilador está vectorizando el código utilizando instrucciones SIMD para procesar múltiples datos a la vez. Pero el algoritmo salta por la memoria leyendo las columnas, la vectorización satura la memoria RAM, haciendo el programa más lento.

Genera el ensamblador con gcc -O3 -march=native -S matmul.c y busca instrucciones vectoriales (vfmadd, vmulps en x86; fmla en ARM).

2. Python puro contra Numpy

Para n = 256, ¿cuál es la razón de tiempos? Para n=256, python puro tardó 0.9495s y Numpy tardó 0.00035s, lo que resulta que Numpy fue más rápido.

Separa dos causas: intérprete contra código compilado, y BLAS optimizado (bloques para caché, vectorización e hilos).

Ya que Python es interpretado pierde mucho más tiempo analizando los tipos de datos dinámicos en cada vuelta de ciclo.

En segunda Numpy delega el cálculo a OPENBLAS, una librería que aplica vectorización y cache blocking, dividiendo las Matrices para que puedan caber en la memoria cache.

3. Paralelismo oculto

Compara numpy y numpy_1hilo para n = 2048 y observa la columna cpu_over_wall. Para n=2048, Numpy sin restricciones tardó 0.0697s, mientras que si forzaba a 1 hilo tardó 0.3215s.

¿Cuántos hilos usó NumPy sin que lo pidieras? Al dividir los tiempos obtenemos un factor de 4.6x, lo que confirma que uso implícitamente entre 5 y 6 hilos, así coincidiendo con los 6 núcleos del procesador.

¿Qué implica para medir una versión "secuencial"? Regla del curso: la línea base V1 del proyecto se mide con un solo hilo (OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1).

Si intentamos usar el Numpy por defecto como línea base secuencial, no sería del todo válido ya que estaríamos comparando el código contra algo que ya está paralelizado.

4. Pico teórico

Estima el pico en doble precisión de tu CPU: $\text{pico} \approx \text{núcleos} \times \text{frecuencia (GHz)} \times \text{FLOP por ciclo por núcleo (tabla siguiente)}$. El pico teórico del Ryzen 5 3600 es: 6 núcleos x 4.20 GHz x 16 FLOP/ciclo = 403.2 GFLOPS.

¿Qué porcentaje del pico alcanzó tu mejor versión en C y cuál NumPy con todos los hilos? La mejor versión de python alcanzó 1.015 GFLOPS y la versión de Numpy con todos los hilos alcanzó 247.17 GFLOPS.

¿Por qué la versión ingenua está tan lejos del pico aun con -O3? Porque el procesador pasa la mayor parte del tiempo inactivo esperando a que la RAM le entregue los datos, por lo que nunca satura las unidades vectoriales.

5. Variabilidad

¿Cuál fue la desviación estándar relativa (desv./promedio) más alta de todas tus mediciones? La desviación estándar relativa más alta ocurrió en la variante de Numpy para n=512, donde el promedio fue de 0.0035s con una desviación de 0.0017s.

¿En qué configuración ocurrió y a qué lo atribuyes? En tamaños de matriz pequeños, ya que el cálculo dura poco que el tiempo que tarda el sistema operativo Ubuntu en crear y destruir los múltiples hilos va generando ruido y haciendo inestable la medición.

6. CPU contra GPU

Para n = 256 y n = 4096, ¿cuál es la razón entre NumPy (CPU de Colab) y CuPy (GPU)?

Para n = 256, la razón (speedup) fue de 10.4x a favor de la GPU (0.0101 s en CPU/0.000097s en GPU).

Para n = 4096, la razón subió drásticamente a 37.1x a favor de la GPU (1.4985s en CPU/0.04036s en GPU).

¿Por qué en n pequeño la GPU no gana? Para Matrices pequeñas, el tiempo que tarda la CPU en preparar los datos, lanzar el kernel (la instrucción) hacia la GPU a través del bus PCI Express, y coordinar la ejecución, es casi tan largo como el cálculo matemático en sí. La GPU no alcanza a saturar sus miles de núcleos con suficiente trabajo para justificar el costo de latencia del lanzamiento (overhead). Con n=4096, el trabajo matemático crece 2n³ haciendo que el tiempo de cálculo opaque por completo el overhead de transferencia, y es ahí donde el paralelismo masivo de la GPU logra beneficiar más.

¿Qué obtuviste al medir sin synchronize() y por qué ese número es falso? Sin sincronizar, los tiempos medidos fueron en tiempos minúsculos casi constantes sin importar el tamaño de la matriz (alrededor de 0.000068 s para n=256 y 0.000071 s para n=4096) Ese número es falso porque el reloj de Python se detuvo inmediatamente después de enviar la orden de ejecución a la tarjeta gráfica (una llamada asíncrona) y no espero a que la GPU realmente

terminara de multiplicar los números. El tiempo medido es solo lo que tardó la CPU en colocar la tarea en la colá de la GPU, no el tiempo de ejecución real.

Sección 6. GPU

```
Fri Oct 2 01:07:23 2026
+-----+
| NVIDIA-SMI 580.82.07                  Driver Version: 580.82.07   CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name            Persistence-M   Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp    Perf       Pwr:Usage/Cap       Memory-Usage  GPU-Util  Compute M.  |
|                                           MIG M.       |
+-----+-----+-----+-----+-----+-----+
|  0   Tesla T4              Off          00000000:00:04:0  Off          0          |
| N/A   34C    P8              9W / 70W         0MiB / 15360MiB    0%      Default  |
|                                           N/A          |
+-----+-----+-----+-----+-----+-----+

Processes:
+-----+-----+-----+-----+-----+-----+
| GPU  GI    CI          PID  Type  Process name                        GPU Memory |
| ID   ID    ID                                   Usage   |
+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+-----+-----+-----+-----+

Build cuda_13.0.r13.0/compiler.36424714_0
Tesla T4 | CuPy 14.0.1
```

Figura 5. Salida de la celda !nvidia-smi.

```
n= 256 | CPU 0.0101 +- 0.0028 s | GPU 0.00097 +- 0.00170 s
| speedup 10.4x GPU | sin sync 0.000068 s (incorrecto)

n= 1024 | CPU 0.0202 +- 0.0001 s | GPU 0.00094 +- 0.00004 s
| speedup 21.5x GPU | sin sync 0.000058 s (incorrecto)

n= 4096 | CPU 1.4985 +- 0.3723 s | GPU 0.04036 +- 0.00827 s
| speedup 37.1x GPU | sin sync 0.000071 s (incorrecto)
```

Figura 6. Salida de la celda comparativa entre CPU y GPU.

Sección 7. Problemas encontrados y soluciones

Se describen a continuación los problemas y sus respectivas soluciones respecto al desarrollo de la práctica.

- Al optarse por una máquina física con Linux nativo en remoto, fue necesario establecer un canal seguro para los tres integrantes. Se integró la máquina anfitriona a una red privada virtual mediante *tailscale*, se configuraron perfiles independientes y se enlazaron con la extensión Remote - SSH de Visual Studio Code permitiendo a cada integrante acceder al entorno de trabajo.
- Durante la integración de los scripts y los primeros resultados experimentales en CSV, un error de sincronización provocó que un integrante realizará un *push* en el repositorio sobrescribiendo sobre la rama principal provocando la pérdida del progreso y desalineación de los historiales en el repositorio. Para su solución, se utilizó *git reflog* y ramas locales para recuperar los commits y archivos aceptados así como la implementación de políticas de protecciones sobre la rama principal *main* mediante *pull requests*.
- Al ejecutar el servidor de desarrollo compartido, existía el riesgo de que sesiones concurrentes de SSH o procesos en segundo plano introdujeron ruido térmico o de planificación elevando la desviación estándar relativa más allá del límite establecido del 10%. Para su solución, se

utilizaron ventanas de ejecución exclusivas para las pruebas mediante *run_all.sh* así como la verificación de que el equipo contará con la disipación adecuada.

Sección 8. Conclusiones

Se pudo llegar a la conclusión de que ningún cálculo realizado con la funciones de *speedup()* es válido si la línea base secuencial no se mide de forma que esta sea válida y reproducible bajo un control del reloj monótonico, descarte de calentamiento y repeticiones. Además, se nota como las banderas *-O3 -march=native* aprovechan extensiones vectoriales SIMD de la CPU lo que eleva bastante los GFLOPs sin la necesidad de implementar paralelismo. Se pudieron detectar paralelismo oculto en las librerías como BLAS o NumPy que son importantes controlar ya que pueden alterar la edición secuencial si no se fija el *OMP_NUM_THREADS=1*. Finalmente, con el uso de CUDA, se notó la importancia de sincronizar antes de leer el reloj para no generar tiempos de encolado dada su naturaleza asíncrona.

Sección 9. Uso de IA generativa

Uso de Gemini (Google) como asistente conversacional utilizado como soporte técnico, conceptual e interpretación de resultados. Se empleó como apoyo en la estructuración del warm-up y la asincronía en GPUs. También, para la síntesis de los requerimientos del formato de la práctica y consultas sobre la inserción y visualización estética de los bloques de código. Ningún análisis de rendimiento ni respuestas fue asumido por la IA, todos los cálculos se obtuvieron y contrastaron manualmente contra los archivos generados en el entorno de Linux alojado en el servidor privado.

Sección 10. Tabla de contribución individual

Nombre	Tareas asignadas	Porcentaje de contribución
Dimas Gonzalez Santiago	Scripts de prueba	40%
	Notebook GPU	
Mendieta Mondragon Cristian	Configuración del entorno	30%
	Código secuencial	
	Automatización	
	Redacción del reporte	
Miranda Ferreyra Uriel	Creación del repositorio	30%
	Integración del repositorio	
	Gestión del repositorio	
	Redacción del reporte	
	Gráficas	

Tabla 3. Contribución por integrante al desarrollo de la práctica.